TPI — Programación 1: Sistema de Control de Inventario

En este trabajo se construye un programa con menú interactivo que controle el inventario de una ferretería, y se explica cómo se pensó en un informe y en un video.

1. De qué trata el trabajo

Una ferretería local necesita digitalizar el control de sus productos para evitar pérdidas de stock. Hoy maneja sus datos de forma manual y requiere un programa que permita gestionar las herramientas a la venta y sus unidades disponibles en tiempo real.

En este trabajo, "en tiempo real" significa que cada operación actualiza el stock en el momento, mientras el programa está en ejecución. Los datos viven en `inventario` y no se guardan al cerrar el programa.

La tarea consiste en construir ese programa. Pero el trabajo no se agota en el código: también pide documentar el diseño de la solución y explicar en un video las decisiones tomadas. Lo que se evalúa es la comprensión de cómo y por qué está construido.

Qué se practica

Estos objetivos de aprendizaje se desprenden de lo que pide el enunciado:

- Representar información real con una lista de diccionarios.
- Dividir un programa en funciones, cada una con una tarea clara.
- Usar condicionales y bucles para controlar el flujo de un programa interactivo.
- Hacer que el programa resista los errores del usuario con `try`/`except` y `raise`.
- Documentar el diseño de la solución.
- Explicar las decisiones tomadas con palabras propias, por escrito y en video.

Vocabulario de este enunciado

El enunciado usa varias palabras para la misma idea, y la palabra "cantidad" para ideas distintas. Esta tabla indica cómo leerlas.

Palabra	Qué significa acá
herramienta, producto	Un elemento del inventario. Se usan como sinónimos.
stock, existencias, unidades disponibles	El número de unidades disponibles de una herramienta. Es lo que guarda la clave `'cantidad'`.
cantidad de herramientas a cargar	Cuántas herramientas distintas se registran en una carga (opción 1). No son unidades.
unidades del movimiento	Las unidades que se restan en una venta o se suman en un ingreso (opción 6).
inventario	La lista de diccionarios donde se guardan todas las herramientas.
catálogo	Las herramientas ya cargadas en `inventario`.
bloque principal	La parte del programa que arranca y repite el menú.

La clave del diccionario se llama `'cantidad'`, pero guarda el stock de la herramienta. No debe confundirse con las otras cantidades de la tabla.

2. Reglas técnicas (restricciones estrictas)

El código debe respetar todas las limitaciones de esta sección. El incumplimiento de cualquiera de las reglas 2.1 a 2.6 limita la calificación a un máximo de 10 sobre 100 (10 %). Es un tope de nota, no un descuento. En la sección 9 se define qué cuenta como incumplimiento de cada regla.

Estas restricciones son condiciones de este trabajo, pensadas para la práctica. No son prohibiciones generales de Python. Conviene leerlas con atención antes de escribir la primera línea.

2.1 Estructuras permitidas

Se puede usar:

- Estructuras secuenciales.
- Condicionales: `if`, `elif`, `else`.
- Bucles: `while`, `for`, con `break` y `continue` si hacen falta.
- Una estructura de selección para el menú: `if`/`elif`/`else`, como en el esquema de la sección 6, o `match`/`case`.
- Definición de funciones con `def` (incluidas funciones auxiliares) y `return`.

Además, el trabajo exige usar `try`/`except` y `raise` para manejar errores (ver 2.4).

Las estructuras de esta lista son las de control y de organización del código. Las funciones de entrada, salida y conversión que usa el esquema de la sección 6 (`input`, `print`, `int`) no cuentan como estructuras y se pueden usar. Con el mismo criterio también se pueden usar `len()`, por ejemplo para saber si `inventario` está vacío; `range()` dentro de un `for`, por ejemplo para repetir la carga de la opción 1; y los f-strings para armar mensajes, como el `print(f'Error: {e}')` del esquema.

Para validar y comparar nombres se pueden usar los métodos de texto de Python, por ejemplo `strip()` y `lower()`. También se puede dividir el programa en varios archivos `.py` propios.

2.2 Estructura de datos

Se debe usar exclusivamente una lista de diccionarios, llamada `inventario`. Cada elemento de la lista representa una herramienta, y cada diccionario debe tener exactamente las claves `'herramienta'` y `'cantidad'`. El inventario se guarda únicamente ahí.

Así se ve esa lista con dos herramientas, 10 martillos y 3 palas:

```
[{'herramienta': 'Martillo', 'cantidad': 10}, {'herramienta': 'Pala', 'cantidad': 3}]
```

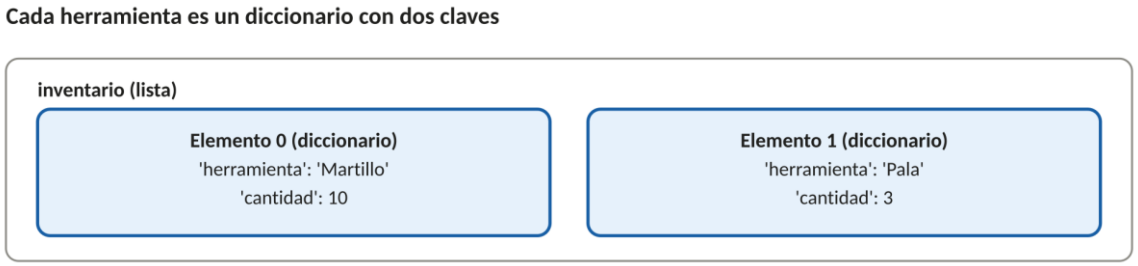


Figura 1. Lista de diccionarios con dos herramientas.

Cada caja interna es una herramienta, y todas tienen las mismas dos claves.

Recorrer `inventario` está permitido. En `for producto in inventario`, la variable `producto` apunta a un elemento que ya pertenece a la lista: no es un diccionario nuevo ni una segunda estructura de datos.

2.3 Modularización obligatoria

El código debe organizarse mediante funciones propias, definidas con `def`. Las opciones 1 a 6 del menú deben resolverse, al menos en su lógica principal, cada una dentro de una función dedicada. La opción 7 puede resolverse en `main()`, terminando el bucle del menú. No se aceptan programas con todo el código en el bloque principal sin modularizar.

Para pensar: una función con una tarea específica es más fácil de leer, probar y corregir. Si hay un error en la consulta de stock, se sabe en qué función buscarlo.

2.4 Manejo de errores

- Se usa `try`/`except` para capturar entradas inválidas. Por ejemplo, texto donde se espera un número.
- Se usa `raise` para lanzar errores con mensaje descriptivo cuando los datos no cumplan las reglas de negocio: nombre vacío, duplicado, stock negativo, etc.

Son dos situaciones distintas. Si el usuario escribe "abc" donde va un número, es una entrada inválida y la captura `try`/`except`. Si escribe un nombre que ya existe, la entrada es un texto válido pero viola una regla de negocio, y se lanza explícitamente con `raise` y un mensaje claro.

Cada pieza hace un trabajo distinto:

- `int()` falla ante una entrada que no se puede convertir a un número entero, por ejemplo "abc" o "3.5". Esa falla ya es un `ValueError`: no hace falta escribir otro `raise`.
- Un condicional comprueba una regla de negocio, por ejemplo que el stock no sea negativo. Si la regla se incumple, `raise` lanza un `ValueError` con un mensaje descriptivo.
- `except` captura el error, pero no repite nada por sí solo. El reintento lo organiza un bucle que vuelve a pedir el dato mientras no sea válido.

`ValueError` es una excepción que ya viene incorporada en Python: no hace falta definir una clase propia para usarla (y en este trabajo las clases están prohibidas, ver 2.5). Se lanza directamente con `raise`, escribiendo el mensaje entre paréntesis. Por ejemplo, con un dato ajeno al inventario:

```
edad = int(input('Edad: '))
if edad < 0:
    raise ValueError('La edad no puede ser negativa.')
```

Ese mensaje es el que después muestra el `except` que captura el error, por ejemplo con `print(f'Error: {e}')`, como en el esquema de la sección 6.

Qué pasa con un error según dónde se capture:

- Si la función no lo captura, el error sale de la función y llega al `except` de `main()`, que muestra el motivo y vuelve al menú.
- Si la función lo captura para reintentar el dato (opción 1), el error no sigue hacia `main()`.
- Si la función lo captura para cambiar o contextualizar el mensaje y después hay que volver al menú, tiene que lanzarlo de nuevo con `raise`. Si solo lo muestra con `print`, `main()` no se entera.

La tabla de la sección 5 es la referencia: dice en qué casos `raise ValueError` es obligatorio y en cuáles alcanza una condición con un mensaje.

Informar que una herramienta no existe o que todavía no hay herramientas cargadas no necesita una excepción: alcanzan una condición y un mensaje claro.

2.5 Prohibiciones

No se permiten:

- Clases.
- Diccionarios individuales fuera de la lista `inventario`. No se crea un diccionario suelto para después agregarlo: el diccionario de una herramienta nueva se arma directamente dentro de `inventario.append(...)`.
- Otras colecciones de datos fuera de `inventario` (listas, diccionarios, conjuntos, colas dobles, etc.). Los resultados se obtienen recorriendo `inventario`.

Sí están permitidas las variables simples (texto y números) y las variables que apuntan a elementos ya existentes de `inventario`.

2.6 Variables globales: prohibidas

Bajo ningún punto de vista se pueden usar variables globales. Los datos compartidos entre funciones, como `inventario`, se pasan como parámetros. Por ejemplo, `inventario` no puede declararse ni usarse de manera global: se declara dentro de una función y se pasa como parámetro a las demás. Las funciones sí pueden tener variables locales y pedir datos al usuario.

Para pensar: con solo mirar la primera línea de una función se sabe de qué datos compartidos depende. Pero pasar `inventario` por parámetro no lo copia, lo comparte: las funciones que cargan o actualizan el stock modifican el mismo inventario que reciben, y el cambio se ve en `main()`.



Figura 2. inventario por parámetro, sin variables globales.

Las funciones que consultan o modifican el inventario lo reciben en su primera línea, y lo que modifican se ve en `main()`, que es quien lo creó. Una función auxiliar que solo valida un valor, por ejemplo un entero positivo, puede recibir únicamente ese valor.

Esto ocurre porque las listas y los diccionarios son tipos mutables: al pasar una lista por parámetro, la función recibe una referencia a esa misma lista, no una copia. Por eso un `append()` hecho dentro de la función se ve después en la función que la llamó:

```
def agregar_nota(notas):
    notas.append(8)

def main():
    notas = [7]
    agregar_nota(notas)
    print(notas)     # muestra [7, 8]
```

No hace falta devolver la lista con `return` para que el cambio se conserve: la función y `main()` trabajan sobre la misma lista. Lo mismo pasa con `inventario`.

3. Diseño (previo al código)

La solución se piensa antes de programar. Lo que se entrega y se evalúa es el diseño documentado junto con el código. Se preparan dos cosas.

1. Un informe de diseño y fundamentación que explique los conceptos aplicados. Cada concepto se vincula con una parte concreta del programa, en lugar de limitarse a definirlo:
   - Listas
   - Diccionarios
   - Funciones
   - Condicionales
   - Bucles
   - Manejo de errores con `try`/`except` y `raise`
2. Un diagrama o esquema que defina el flujo de las operaciones principales. Incluye el menú y los recorridos de carga, validación de datos y actualización de stock. Va dentro del informe PDF.

El informe también incluye el lenguaje, el entorno y los recursos de desarrollo utilizados, con la investigación realizada sobre ellos. Alcanza con un párrafo breve por cada uno: qué es, qué versión se usó y por qué se eligió. Si el programa tiene más de un archivo `.py`, se indica en el informe cuál se ejecuta para iniciarlo.

El informe se entrega en PDF dentro del ZIP (ver sección 7). Como referencia de cómo puede verse un esquema de flujo, en la sección 4 hay un gráfico del menú; el del informe debe reflejar el diseño propio.

4. Qué debe hacer el programa

El programa debe presentar un menú interactivo persistente con siete opciones. Persistente quiere decir que, después de cada operación, el menú vuelve a mostrarse hasta que el usuario elige la opción 7 para salir.

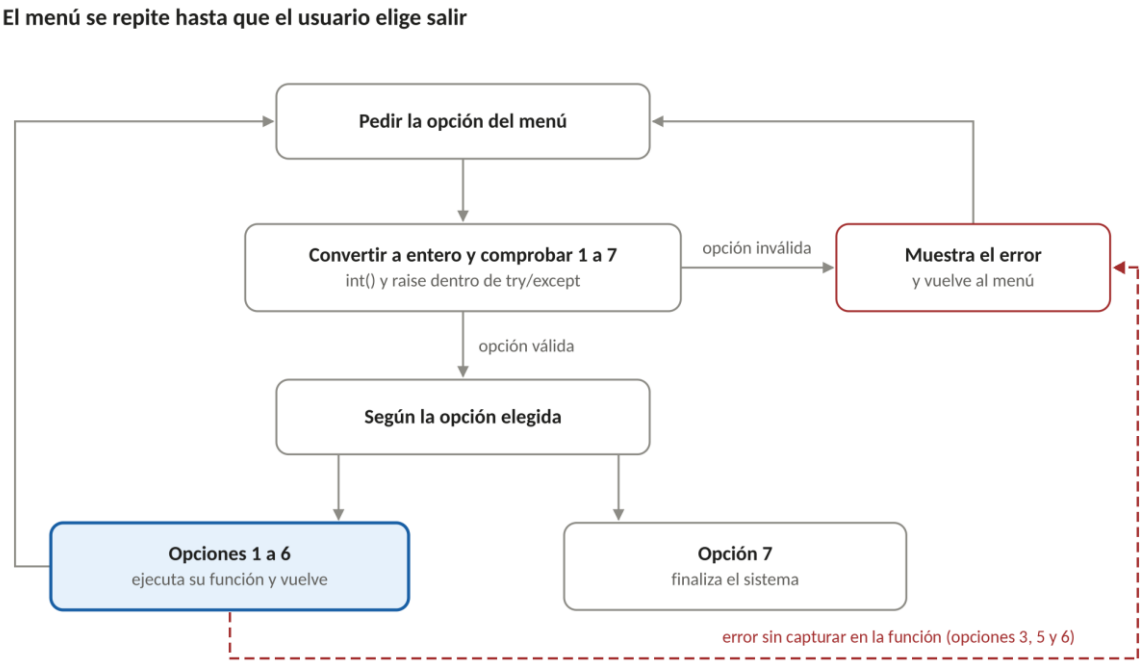


Figura 3. Flujo del menú.

Si la opción es inválida, el programa muestra el error y vuelve al menú. Con las opciones 1 a 6 ejecuta la función correspondiente y también vuelve al menú. Si una de esas funciones deja salir un error sin capturarlo (opciones 3, 5 y 6), también llega al `except` de `main()`: se muestra el error y se vuelve al menú. Solo la opción 7 termina el programa.

Reglas generales ante datos inválidos

Un dato inválido nunca deja datos a medias. Cada herramienta se incorpora a `inventario` solo cuando su nombre y su stock son válidos, y un intento inválido no modifica ningún stock. Si una carga múltiple falla en una herramienta, se conservan las que ya se agregaron.

Todo nombre que se ingresa (opciones 1, 3, 5 y 6) se valida antes de usarse: un nombre vacío, o formado solo por espacios, lanza `ValueError`.

Opción 1 — Carga de herramientas con existencias iniciales

Sirve para registrar las herramientas que se pondrán a la venta. Es la carga múltiple: varias herramientas en una sola ejecución de la opción. El flujo es este:

1. El sistema pregunta cuántas herramientas distintas se van a cargar. Debe ser un número entero mayor que 0. Si no se puede convertir a entero, `int()` produce el `ValueError`; si es 0 o negativo, se lanza un `ValueError` con `raise`. En los dos casos el error se captura dentro de la misma función, se informa y se vuelve a pedir el dato.
2. Por cada una, solicita inmediatamente su nombre y luego su stock inicial. No hay una opción separada para las existencias.
3. Si el nombre está vacío o duplicado, se lanza un error con `raise`, se captura dentro de la misma función y se vuelve a pedir el nombre.
4. Si el stock inicial no es un número entero (`int()` produce el `ValueError`) o es negativo (se lanza un `ValueError` con `raise`), el error se captura dentro de la misma función, se informa y se vuelve a pedir solo el stock, conservando el nombre válido. El 0 es un stock inicial válido.
5. La herramienta se agrega a `inventario` recién cuando el nombre y el stock son válidos. Recién ahí cuenta como cargada.
6. Se repite hasta completar la cantidad indicada por el usuario antes de iniciar la carga.

Si la opción 1 se ejecuta más de una vez, las nuevas herramientas se agregan a las que ya estaban cargadas, y el control de duplicados considera todas.

Opción 2 — Visualización de inventario

Muestra el listado completo de herramientas junto a su stock actual, recorriendo la lista `inventario`. Si todavía no hay herramientas cargadas, lo informa con un mensaje.

Opción 3 — Consulta de stock

Busca una herramienta por su nombre y muestra cuántas unidades hay disponibles. El orden es este:

1. Si todavía no hay herramientas cargadas, lo informa con un mensaje y vuelve al menú, sin pedir ningún dato.
2. Pide el nombre y lo valida: un nombre vacío lanza `ValueError`, se muestra el motivo y se vuelve al menú.
3. Busca la herramienta. Si no está en el catálogo, lo informa claramente con un mensaje. Si está, muestra su stock.

Opción 4 — Reporte de agotados

Lista únicamente aquellos productos cuyo stock sea igual a cero. Si todavía no hay herramientas cargadas, lo informa con un mensaje. Si hay herramientas cargadas pero ninguna tiene stock cero, lo informa con otro mensaje distinto.

Opción 5 — Alta de nuevo producto

Agrega una nueva herramienta al final de `inventario`, junto con su stock inicial. Primero pide el nombre y lo valida; recién si el nombre es válido pide el stock. Así no se pide el stock de un producto que ya fue rechazado. En caso de nombre vacío o duplicado, o de stock negativo, se usa `raise` para lanzar un `ValueError` descriptivo. Si el stock no se puede convertir a un número entero, `int()` ya produce el `ValueError`. En ambos casos el error sale de la función sin ser capturado, lo recibe el `except` de `main()` y se vuelve al menú principal mostrando el motivo, sin agregar nada.

No se exige un mensaje personalizado para las conversiones que fallan. Si se quiere agregar, se captura el error y se lo lanza de nuevo con `raise` (ver 2.4).

Conviene notar la diferencia con la opción 1. En la opción 1, el error se captura dentro de la función y se vuelve a pedir el dato. En la opción 5, el error sale de la función y la vuelta al menú la resuelve `main()`.

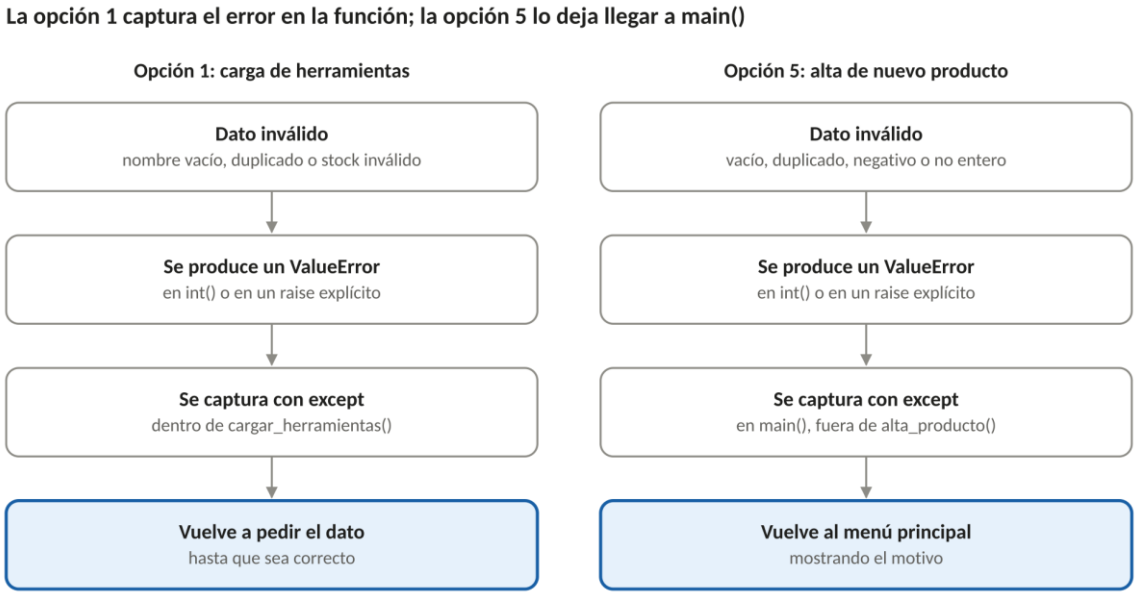


Figura 4. Dónde se captura el error en la opción 1 y en la opción 5.

Opción 6 — Actualización de stock (venta / ingreso)

Permite modificar el stock de una herramienta existente:

- Venta: disminuye el stock tras validar que hay unidades suficientes. Se usa `raise` si el stock resultante sería negativo.
- Ingreso: aumenta el stock por reposición de mercadería.

El flujo es este, en este orden:

1. Si todavía no hay herramientas cargadas, lo informa con un mensaje y vuelve al menú, sin pedir ningún dato.
2. Pide el nombre y lo valida: un nombre vacío lanza `ValueError`.
3. Comprueba que la herramienta exista. Si no existe, lo informa con un mensaje.
4. Pide el tipo de movimiento, venta o ingreso. Las dos alternativas se pueden ofrecer como números o como texto, pero solo se aceptan esas dos: cualquier otra entrada muestra un mensaje. La entrada se

compara como texto (por ejemplo `'1'` y `'2'`, o `'venta'` e `'ingreso'`), sin convertirla con `int()`: así una entrada inválida se resuelve con una condición y un mensaje, como indica la sección 5.

5. Pide las unidades del movimiento y las valida: deben ser un número entero mayor que 0.
6. Si es una venta, comprueba que haya unidades suficientes.
7. Recién entonces modifica el stock.

Si falla cualquiera de los pasos 1 a 6, se muestra el motivo, se vuelve al menú y el stock no se modifica. Así no se piden unidades para una herramienta inexistente ni para un movimiento ya rechazado.

Opción 7 — Salir

Finaliza la ejecución del sistema. No necesita una función propia: se resuelve en `main()` cuando termina el bucle del menú.

5. Criterios de validación (robustez)

El sistema debe ser "a prueba de errores" comunes. Esta tabla es la referencia común para el texto, los gráficos, las pruebas y la evaluación: en cada situación indica el mecanismo exigido. Las reglas de negocio se protegen con `raise ValueError`. Las entradas que no se pueden convertir a un número entero producen un `ValueError` en `int()` y se capturan con `try`/`except`. Las situaciones informativas se resuelven con una condición y un mensaje claro, sin excepción.

Situación	Cómo se resuelve
Nombre de herramienta vacío (o de solo espacios) o duplicado.	Lanzar `ValueError` con mensaje descriptivo.
Cantidad, stock o unidades que no se pueden convertir a un número entero (por ejemplo "abc" o "3.5").	Capturar con `try`/`except`: dentro de la función en la opción 1, para reintentar, y en `main()` en las opciones 5 y 6. Ver "Qué pasa tras cada error".
Stock inicial negativo (el 0 es válido).	Lanzar `ValueError`.
Cantidad de herramientas a cargar, o unidades de una venta o ingreso, que no son mayores que 0.	Lanzar `ValueError`.
Venta que supera el stock disponible.	Lanzar `ValueError`.
Operar sobre una herramienta que no existe, o sin herramientas cargadas.	Condición y mensaje claro, sin modificar nada.
Tipo de movimiento distinto de venta o ingreso.	Condición y mensaje claro, sin modificar el inventario.
Opción del menú que no se puede convertir a un número entero.	Capturar con `try`/`except` en `main()`.
Opción del menú entera pero fuera del rango de 1 a 7.	Lanzar `ValueError`.

Valores aceptados

Dato	Condición requerida
Cantidad de herramientas a cargar	Entero mayor que 0.
Stock inicial	Entero mayor o igual que 0.
Unidades de venta o ingreso	Entero mayor que 0.
Opción del menú principal	Entero entre 1 y 7.

Un valor decimal, como `3.5`, no es un entero: `int('3.5')` falla y la entrada se rechaza, sin transformarla en otro valor.

Cómo se comparan los nombres: un nombre formado solo por espacios cuenta como vacío. Para detectar duplicados y para buscar una herramienta (opciones 3 y 6), "Martillo" y "martillo" son la misma herramienta, y se ignoran los espacios al inicio y al final. El nombre se guarda en `inventario` sin los espacios del inicio y del final (con `strip()`), respetando las mayúsculas que escribió el usuario.

Qué pasa tras cada error

Operación	Dato inválido	Qué pasa
Opción 1	Nombre vacío o duplicado.	Se vuelve a pedir el nombre.
Opción 1	Stock inicial inválido.	Se vuelve a pedir solo el stock, conservando el nombre.
Opción 1	Cantidad de herramientas a cargar inválida.	Se vuelve a pedir ese dato.
Opción 3	Nombre vacío.	Se muestra el motivo y se vuelve al menú.
Opción 5	Cualquier dato inválido.	Se muestra el motivo y se vuelve al menú, sin agregar nada.
Opción 6	Cualquier dato inválido, o venta que supera el stock.	Se muestra el motivo y se vuelve al menú, sin modificar el stock.

6. Ejemplo de estructura mínima esperada

Este esquema ilustra la organización modular requerida. No es el único diseño válido, pero sí la referencia mínima aceptable.

```
def cargar_herramientas(inventario): ...
def mostrar_inventario(inventario): ...
def consultar_stock(inventario): ...
def reporte_agotados(inventario): ...
def alta_producto(inventario): ...
def actualizar_stock(inventario): ...

def main():
    inventario = []
    opcion = 0
    while opcion != 7:
        try:
            opcion = int(input('Seleccione una opción: '))
            if opcion == 1:
                cargar_herramientas(inventario)
            elif opcion == 2:
                mostrar_inventario(inventario)
            # ... etc.
            elif opcion != 7:
                raise ValueError('Opción fuera de rango (1 a 7).')
        except ValueError as e:
            print(f'Error: {e}')

main()
```

Qué mirar en este esquema:

- `inventario` se crea dentro de `main()` y se pasa como parámetro a cada función. No hay variables globales.
- El menú vive en un `while` que se repite hasta que `opcion` vale 7. La opción 7 no necesita una función propia.
- El `except ValueError` de `main()` atiende lo que llega hasta ahí: la entrada que no se puede convertir a entero, que hace fallar a `int()`; la opción fuera de rango, que lanza el `raise`; y los errores que salen sin capturar de las funciones de las opciones 3, 5 y 6.
- La opción 1 captura por su cuenta los errores que provocan un reintento: un bucle vuelve a pedir el dato. Si esos errores llegaran hasta `main()`, el programa volvería al menú y no continuaría con la carga pendiente.
- Las funciones están escritas con `...`: completarlas es parte del trabajo.
- El esquema no muestra el texto del menú con las siete opciones. Imprimirlo antes de pedir la opción también es parte del trabajo.
- Al completar las opciones 3 a 6, se agregan antes de la última rama `elif opcion != 7`.

7. Qué se entrega

A continuación se detallan los componentes y requisitos de la entrega. La sección 9 distingue las condiciones de admisión de los aspectos que se califican por su calidad y grado de cumplimiento.

7.1 Archivo comprimido

Se entrega un único archivo `.zip` nombrado como `Apellido_Nombre_P1.zip`.

7.2 Contenido del ZIP

1. Todos los archivos con extensión `.py`, para garantizar la correcta ejecución del programa.
2. Informe PDF con el diseño de la solución (conceptos aplicados y diagrama de flujo) y con el lenguaje, el entorno y los recursos de desarrollo utilizados (explicado en la sección 3).

7.3 Video explicativo

Se debe realizar un video de entre 5 y 10 minutos de duración. La duración es una condición de entrega. El link del video, subido a YouTube o Google Drive, se coloca en el comentario de la entrega, con permisos de acceso para el docente. Qué explicar y cómo grabarlo está en la sección 8.

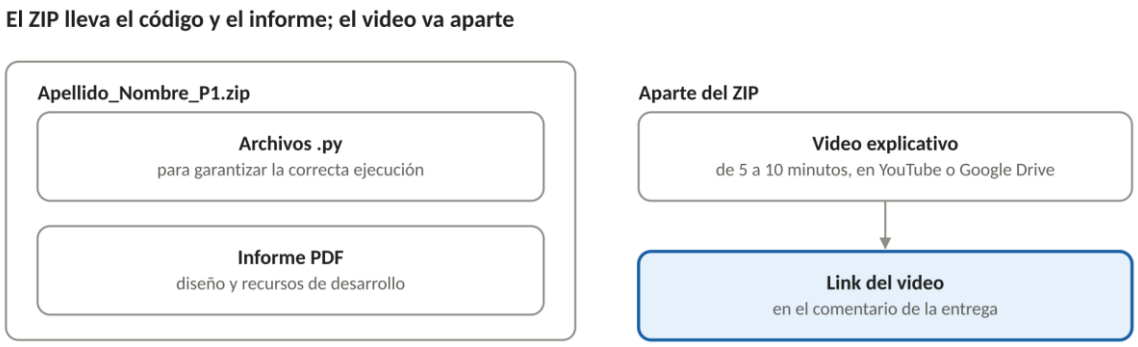


Figura 5. Contenido de la entrega: ZIP y video.

7.4 Restricciones de entrega

- No se califican archivos subidos en los comentarios de la tarea de entrega.
- No se califican archivos subidos fuera de la plataforma institucional.
- No se califica código enviado mediante enlaces directos a repositorios (GitHub, Google Drive, etc.).
- Estas restricciones no alcanzan al link del video, que se coloca en el comentario de la entrega como indica 7.3.

Qué condiciones de entrega deciden si el trabajo se evalúa, qué incumplimientos limitan la nota y cómo se califica el resto está en la sección 9.

8. El video explicativo

El video dura entre 5 y 10 minutos y es la oportunidad de demostrar que se entiende el propio programa. Por eso el enunciado pide explicar y no leer el código.

Qué se explica

- Cómo se estructuró el código, la lógica detrás del sistema y el rol de cada función. No se lee el código: se explica y se demuestra comprensión.
- Por qué se eligieron las estructuras de control y el diseño modular para cada requerimiento.
- Cómo se aplican `try`/`except` y `raise` en el programa.
- Una breve demostración del funcionamiento: un caso de éxito en cada opción del menú y, donde hay validación, un caso de error.

Secuencia de demostración

Se sigue el orden del menú:

1. Opción 1: una carga con éxito y un error de validación, por ejemplo un nombre duplicado o un stock inválido. Se carga una herramienta con stock positivo y otra con stock 0: así se puede mostrar el reporte de agotados antes de llegar a la opción 6.
2. Opción 2: el listado.
3. Opción 3: una consulta con éxito y un error (nombre vacío). Se muestra también una herramienta que no existe.
4. Opción 4: el reporte de agotados.
5. Opción 5: un alta con éxito y un error de validación.
6. Opción 6: una venta y un ingreso con éxito, y un error, por ejemplo una venta que supera el stock.
7. Opción 7: la salida del sistema.

Las opciones 2, 4 y 7 no tienen un caso de error de validación propio, así que en ellas alcanza con el caso de éxito. Una demostración breve de cada caso deja tiempo para lo más importante: explicar el razonamiento.

Los datos se pueden preparar antes de grabar, y las decisiones se pueden comentar mientras se ejecuta cada caso. No hace falta explicar todo el código primero y después volver a narrar cada operación.

Condiciones de grabación

- Se usa cámara para mostrarse a cuadro.
- Se muestra el código en Visual Studio Code, con fondo claro y letra grande.

El link del video, subido a YouTube o Google Drive, va en el comentario de la entrega, con permisos de acceso para el docente (ver 7.3).

9. Cómo se evalúa

El trabajo se controla en tres instancias, siempre en este orden. Cada defecto se juzga en una sola: la primera que lo alcanza.

Instancia	Qué se controla	Si no se cumple
1. Entrega	Que el ZIP tenga el nombre indicado y contenga los archivos `.py` y un informe PDF. Que el link del video funcione con permisos para el docente y que el video dure entre 5 y 10 minutos. Que se respeten las restricciones de 7.4.	El trabajo no se evalúa.
2. Reglas técnicas	Que se respete cada una de las reglas 2.1 a 2.6, según la tabla de abajo.	Calificación máxima de 10 sobre 100.
3. Calidad y cumplimiento	Los cinco criterios que figuran más abajo, divididos en ocho aspectos con puntaje (total: 100 puntos).	La nota es sobre base 100.

Qué cuenta como incumplimiento de cada regla técnica

Regla	Se incumple cuando
2.1 Estructuras permitidas	Se usan estructuras de control u organización fuera de las permitidas. No cuentan como estructuras, y están permitidos: `input`, `print`, `int`, `len`, `range`, los f-strings, `strip()`/`lower()` y dividir el programa en varios archivos `.py` propios (ver 2.1).
2.2 Estructura de datos	El inventario no es una lista de diccionarios llamada `inventario`, con exactamente las claves `'herramienta'` y `'cantidad'`.
2.3 Modularización	Alguna de las opciones 1 a 6 no tiene una función propia, o el programa está todo en el bloque principal.
2.4 Manejo de errores	El programa no usa `try`/`except`, o no usa `raise`.
2.5 Prohibiciones	Se usan clases, diccionarios sueltos u otras colecciones de datos fuera de `inventario`.
2.6 Variables globales	Se usa alguna variable global.

El tope de la instancia 2 no reemplaza a la nota: la instancia 3 se aplica igual y el resultado no puede superar 10. Una validación puntual que falta o está mal resuelta no activa el tope: reduce el nivel de logro del criterio de validaciones. Del mismo modo, un informe al que le falta el diagrama no vuelve la entrega no evaluable: la instancia 1 solo controla que el PDF esté, y lo que le falta se considera en el criterio del informe.

Criterios de calidad y cumplimiento

Son cinco, se evalúan por separado y suman 100 puntos:

- Funcionamiento de las siete opciones del menú (sección 4): 25 puntos.
- Validaciones y recuperación de errores (sección 5): 25 puntos.
- Organización y legibilidad del código: funciones propias y nomenclatura `snake_case`. Por ejemplo, `consultar_stock` y no `consultarStock`. 10 puntos.
- Informe de diseño y fundamentación (sección 3): 25 puntos, repartidos en conceptos aplicados vinculados con el programa (12), diagrama de flujo (8) y lenguaje, entorno y recursos utilizados (5).
- Video (sección 8): 15 puntos, repartidos en explicación (8) y demostración con las condiciones de grabación (7).

Cada aspecto se valora con uno de cinco niveles, que otorgan un porcentaje de su puntaje: Excelente (100 %), Bueno (75 %), Satisfactorio (50 %), Insuficiente (25 %) y No presentado (0 %). Esto es lo que significa cada nivel:

Aspecto (puntos)	Excelente (100 %)	Bueno (75 %)	Satisfactorio (50 %)	Insuficiente (25 %)
Funcionamiento (25)	Las siete opciones existen y hacen todo lo que pide la sección 4: carga múltiple (una segunda ejecución conserva lo ya cargado), listado, consulta, reporte de agotados (con un mensaje propio cuando ninguna herramienta tiene stock cero), alta, venta/ingreso en el orden de pasos indicado y salida. El menú es persistente y solo la opción 7 termina el programa.	Las siete opciones funcionan, pero una sola omite un detalle de la sección 4 (p. ej., el reporte no distingue el caso sin agotados, o la opción 6 no sigue el orden de pasos indicado).	Las siete opciones existen y se pueden usar, pero dos o más omiten parte de lo que pide la sección 4.	Falta alguna opción, o alguna no se puede usar (p. ej., no se pueden cargar herramientas).

Aspecto (puntos)	Excelente (100 %)	Bueno (75 %)	Satisfactorio (50 %)	Insuficiente (25 %)
Validaciones y recuperación de errores (25)	Todas las situaciones de la sección 5 se resuelven con el mecanismo exigido: `raise ValueError` para reglas de negocio (nombre vacío/duplicado, stock negativo, cantidades ≤ 0, venta mayor al stock, opción fuera de rango), `try/except` para conversiones, y condición + mensaje para casos informativos (inventario vacío, herramienta inexistente, tipo de movimiento inválido). La opción 1 reintenta dentro de la función (conserva el nombre válido); las opciones 3, 5 y 6 vuelven al menú vía main(). Nombres comparados sin mayúsculas ni espacios extremos.	Una sola situación de la sección 5 no se resuelve como indican sus tablas (p. ej., no ignora mayúsculas al detectar duplicados), sin dejar datos a medias ni modificar el stock.	Dos o más situaciones de la sección 5 no se resuelven como indican sus tablas, pero ningún dato inválido deja datos a medias ni modifica el stock.	Un dato inválido deja datos a medias o modifica el stock, o hace que el programa se interrumpa.
Organización y legibilidad del código (10)	Cada función hace una tarea clara y todo el código usa `snake_case`.	Una sola función mezcla varias tareas, o algún nombre aislado no usa `snake_case`.	Varias funciones mezclan tareas, o varios nombres no usan `snake_case`.	No se distingue qué hace cada función, o el código es difícil de leer.
Informe: conceptos aplicados (12)	Explica listas, diccionarios, funciones, condicionales, bucles y try/except/raise, vinculando cada concepto con una parte concreta de su programa.	Explica todos los conceptos y la mayoría vinculados con el programa, con alguna omisión menor.	Explica el diseño concreto pero omite algún concepto, o las vinculaciones son escasas.	Solo reproduce definiciones generales y no permite comprender la solución presentada.
Informe: diagrama de flujo (8)	Diagrama propio y claro del menú y de los recorridos de carga, validación de datos y actualización de stock, coherente con el código.	Diagrama correcto pero omite alguno de los recorridos o tiene pequeñas incoherencias con el código.	Diagrama parcial o genérico (p. ej., copia el del enunciado sin reflejar su diseño).	Diagrama confuso o sin relación con el programa.

Aspecto (puntos)	Excelente (100 %)	Bueno (75 %)	Satisfactorio (50 %)	Insuficiente (25 %)
Informe: lenguaje, entorno y recursos (5)	Un párrafo por cada recurso (qué es, versión usada y por qué lo eligió). Si hay varios .py, indica cuál se ejecuta para iniciar.	Menciona los recursos con alguna omisión (versión o justificación).	Solo enumera los recursos, sin investigación ni justificación.	Información incorrecta o mínima.
Video: explicación (8)	Explica, sin leer el código, la estructura, el rol de cada función, por qué eligió las estructuras de control y el diseño modular, y cómo aplica try/except y raise.	Explica con claridad, pero omite o trata superficialmente alguno de esos puntos.	Omite varios puntos o la explicación es superficial.	Lee el código sin explicarlo.
Video: demostración y condiciones de grabación (7)	Sigue la secuencia del menú de la sección 8: éxito en cada opción y un caso de error en las opciones 1, 3, 5 y 6. En la opción 1 carga una herramienta con stock positivo y otra con stock 0; en la 3 muestra además una herramienta inexistente; en la 6 muestra una venta y un ingreso con éxito. Se muestra a cámara, con el código en VS Code, fondo claro y letra grande.	Demostración casi completa: falta uno de esos casos o una condición de grabación.	La demostración queda incompleta: faltan varias opciones o varios de esos casos.	No muestra el funcionamiento del programa.

No presentado (0 %) se aplica cuando ese aspecto falta por completo en la entrega, por ejemplo un informe sin diagrama de flujo.

Escala de la nota: 90 a 100 puntos, Sobresaliente; 70 a 89, Notable; 50 a 69, Aprobado; 0 a 49, Reprobado. La nota es sobre base 100 y, si se aplicó el tope de la instancia 2, no puede superar 10 puntos.

10. Lista de control antes de entregar

Esta lista de control solo reúne requisitos de las secciones anteriores. Conviene repasarla antes de subir el trabajo. Las pruebas manuales que siguen traducen esos mismos requisitos a casos concretos.

- ☐ Se usan solo las estructuras permitidas y una lista de diccionarios con las claves `'herramienta'` y `'cantidad'`.
- ☐ No hay clases, ni otras colecciones de datos fuera de `inventario`, ni diccionarios sueltos: el de una herramienta nueva se arma dentro de `inventario.append(...)`.

- ☐ No hay variables globales, y `inventario` se declara dentro de una función y se pasa como parámetro.
- ☐ Las opciones 1 a 6 se resuelven cada una en una función propia, y la opción 7 en `main()`.
- ☐ Se usan `try`/`except` y `raise` en los casos que indica la tabla de la sección 5.
- ☐ Un dato inválido no deja datos a medias: no se agregan herramientas incompletas ni se modifica el stock.
- ☐ Se probaron todos los casos de la tabla de pruebas manuales.
- ☐ El código es legible y usa `snake_case`.
- ☐ El informe PDF incluye los conceptos aplicados, el diagrama de flujo y los recursos de desarrollo utilizados.
- ☐ El ZIP se llama `Apellido_Nombre_P1.zip` y contiene los archivos `.py` y el informe PDF.
- ☐ El video dura entre 5 y 10 minutos, cumple lo pedido en la sección 8 y su link está en el comentario de la entrega, con permisos de acceso para el docente.

Pruebas manuales

Estos casos sirven para comprobar el programa. Cada caso se prepara de forma independiente: se parte del estado inicial indicado. Cuando una prueba enumera varias entradas, cada una se ejecuta por separado. El resultado esperado es el que define este enunciado.

Prueba	Estado inicial	Resultado esperado
Ingresar `abc` como opción del menú.	Cualquiera.	Muestra el error y vuelve a pedir una opción.
Elegir la opción 9 en el menú.	Cualquiera.	Muestra el error de rango y vuelve a pedir una opción.
Opción 1: ingresar `0`, `-1`, `abc` o `3.5` como cantidad de herramientas a cargar.	Inventario vacío.	Informa el error y vuelve a pedir ese dato.
Opción 1: ingresar un nombre vacío o de solo espacios.	Inventario vacío.	Se vuelve a pedir el nombre.
Opción 1: cargar `martillo`.	`Martillo` cargado.	Se rechaza como duplicado y se vuelve a pedir el nombre.
Opción 1: cargar `  Martillo  ` con espacios al inicio y al final.	`Martillo` cargado.	Se rechaza como duplicado.
Opción 1: cargar `Pala` y escribir `-2` como stock inicial.	`Martillo` cargado.	Se vuelve a pedir solo el stock. `Pala` no se incorpora hasta que el stock sea válido.
Opción 1: cargar `Pala` y escribir `abc` como stock inicial.	`Martillo` cargado.	Se vuelve a pedir solo el stock, conservando el nombre `Pala`.
Opción 1: cargar `Pala` y escribir `3.5` como stock inicial.	`Martillo` cargado.	Se rechaza y se vuelve a pedir el stock. Los datos existentes no cambian.

Prueba	Estado inicial	Resultado esperado
Opción 1: cargar 3 herramientas nuevas; para la segunda, escribir `abc` como stock y después un valor válido.	Inventario vacío.	Se vuelve a pedir solo el stock de la segunda, conservando su nombre. Al final se agregan exactamente 3 herramientas nuevas.
Opción 1: cargar `Pala` con stock 0.	`Martillo` cargado.	Se acepta y `Pala` aparece en el reporte de agotados.
Opción 1: ejecutarla por segunda vez, con herramientas nuevas.	`Martillo` cargado.	Conserva `Martillo` y agrega las nuevas herramientas, sin duplicados.
Opción 2: mostrar el inventario.	Inventario vacío.	Informa que todavía no hay herramientas cargadas.
Opción 3: consultar stock.	Inventario vacío.	Informa que todavía no hay herramientas cargadas y vuelve al menú, sin pedir ningún dato.
Opción 3: consultar `MARTILLO`.	`Martillo` con 10.	Encuentra la herramienta y muestra 10.
Opción 3: consultar `martillo` con espacios al inicio y al final.	`Martillo` con 10.	Encuentra la herramienta y muestra 10.
Opción 3: consultar con un nombre vacío.	`Martillo` cargado.	Muestra el motivo y vuelve al menú.
Opción 3: consultar una herramienta que no está cargada, por ejemplo `Pala`.	`Martillo` cargado.	Informa que no se encuentra en el catálogo.
Opción 4: reportar agotados.	Inventario vacío.	Informa que todavía no hay herramientas cargadas.
Opción 4: reportar agotados.	`Martillo` con 10 y `Pala` con 0.	Muestra solamente `Pala`.
Opción 4: reportar agotados.	Todas con stock positivo.	Informa que ninguna herramienta tiene stock cero.
Opción 5: alta con nombre vacío.	`Martillo` cargado.	Muestra el motivo y vuelve al menú, sin pedir el stock. No queda ningún registro incompleto.
Opción 5: alta de `Martillo`.	`Martillo` cargado.	Se rechaza como duplicado, sin pedir el stock: muestra el motivo y vuelve al menú. No queda ningún registro incompleto.
Opción 5: alta de `Pala` con stock `-2`.	`Martillo` cargado.	Muestra el motivo y vuelve al menú. No queda ningún registro incompleto.
Opción 5: alta de `Pala` con stock `3.5`.	`Martillo` cargado.	Muestra el motivo y vuelve al menú. No queda ningún registro incompleto.

Prueba	Estado inicial	Resultado esperado
Opción 5: alta de `Pala` con stock `abc`.	`Martillo` cargado.	Muestra el motivo y vuelve al menú. No queda ningún registro incompleto.
Opción 5: alta de `Pala` con stock 3.	`Martillo` cargado.	`Pala` se agrega al final de `inventario`.
Opción 5: alta de `Pala` con stock 0.	`Martillo` cargado.	Se acepta: `Pala` se agrega al final de `inventario` y aparece en el reporte de agotados.
Opción 6: actualizar stock.	Inventario vacío.	Informa que todavía no hay herramientas cargadas y vuelve al menú, sin pedir ningún dato.
Opción 6: actualizar `Pala`.	`Martillo` cargado.	Informa que no existe y vuelve al menú. No se piden unidades y el inventario no cambia.
Opción 6: ingresar un nombre vacío.	`Martillo` cargado.	Muestra el motivo y vuelve al menú. No se piden unidades y el inventario no cambia.
Opción 6: nombre `  Martillo  ` con espacios al inicio y al final, y venta de 2 unidades.	`Martillo` con 10.	Encuentra el registro existente y su stock queda en 8.
Opción 6: nombre `Martillo` y un tipo de movimiento que no es venta ni ingreso.	`Martillo` con 10.	Lo rechaza y no modifica el inventario.
Opción 6: en una venta, ingresar `0`, `-1`, `abc` o `3.5` como unidades.	`Martillo` con 10.	Rechaza el movimiento. El stock sigue en 10.
Opción 6: en un ingreso, ingresar `0`, `-1`, `abc` o `3.5` como unidades.	`Martillo` con 10.	Rechaza el movimiento. El stock sigue en 10.
Opción 6: vender 3 unidades.	`Martillo` con 3.	El stock queda en 0.
Opción 6: vender 4 unidades.	`Martillo` con 3.	Informa el error y el stock sigue en 3.
Opción 6: ingresar 2 unidades.	`Martillo` con 0.	El stock queda en 2 y deja de aparecer en el reporte de agotados.